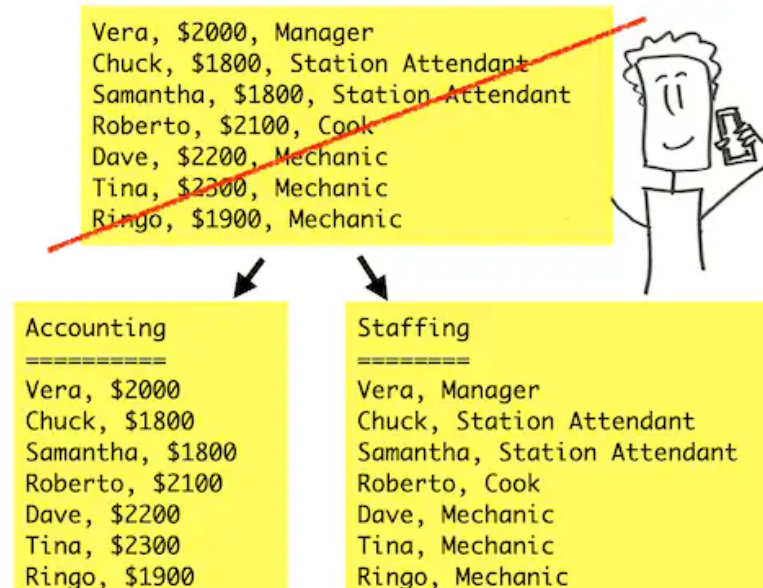


Encapsulation and Dependency Injection

New requirements

- Mr. Star likes the new program so much that he wants to create a new report.

- This time, he wants two different reports: one for Accounting and another for Staffing.



Requirements Version 4

- Mr. Star (the **employer**) delegates: an **account manager** handles payroll, a **staff manager** handles staffing.

- Both managers report to him.

Epic requirement:
As an "employer,"
I want to "generate reports,"
so that "I can manage my employees."

Sub requirement 1:
As an "account manager,"
I want to "have an Accounting report including name and salary,"
so that "I can track monthly salary payments."

Sub requirement 2:
As a "staff manager,"
I want to "have a Staffing report including name and job title,"
so that "I can track my staff."

Two Methods Solution

To solve this problem, we need to make two methods.

```
def print_accounting_report():
```

```
Accounting
=====
Vera, $2000
Chuck, $1800
Samantha, $1800
Roberto, $2100
Dave, $2200
Tina, $2300
Ringo, $1900
```

```
def print_staffing_report():
```

```
Staffing
=====
Vera, Manager
Chuck, Station Attendant
Samantha, Station Attendant
Roberto, Cook
Dave, Mechanic
Tina, Mechanic
Ringo, Mechanic
```

Implementation

```
1 def print_accounting_report():
2     print("Accounting")
3     print("=====")
4     for e in employees:
5         print(f"{e.name}, ${e.salary}")
6
7 def print_staffing_report():
8     print("Staffing")
9     print("=====")
10    for e in employees:
11        print(f"{e.name}, {e.job_title}")
12
13 print_accounting_report()
14 print() # empty line
15 print_staffing_report()
```

```
Accounting
=====
Vera, $2000
Chuck, $1800
Samantha, $1800
Roberto, $2100
Dave, $2200
Tina, $2300
Ringo, $1900

Staffing
=====
Vera, Manager
Chuck, Station Attendant
Samantha, Station Attendant
Roberto, Cook
Dave, Mechanic
Tina, Mechanic
Ringo, Mechanic
```

Problem: Name Collision

- We hire Chuck Rainey, who is a new Station Attendant.
- We cannot distinguish this Chuck from Chuck the mechanic.

Chuck Rainey



NAME	SALARY	JOB TITLE
Vera	2000	Manager
Chuck	1800	Station Attendant
Samantha	1800	Station Attendant
Roberto	2100	Cook
Dave	2200	Mechanic
Tina	2300	Mechanic
Ringo	1900	Mechanic
Chuck	1800	Mechanic



Another Chuck???

Spreadsheet Update Needed

- We need to change the spreadsheet to include first and last names.

FIRST NAME	LAST NAME	SALARY	JOB TITLE
Vera	Schmidt	2000	Manager
Chuck	Norris	1800	Station Attendant
Samantha	Carrington	1800	Station Attendant
Roberto	Jacketti	2100	Cook
Dave	Dreißig	2200	Mechanic
Tina	River	2300	Mechanic
Ringo	Rama	1900	Mechanic
Chuck	Rainey	1800	Mechanic

Requirements Version 5

- We also need to update the requirements.
- The epic stays the same — only the sub-requirements change from `name` to `first name, last name`:

Sub requirement 1:

As an "account manager,"

I want to "have an Accounting report including first name, last name, and salary,"
so that "I can track monthly salary payments."

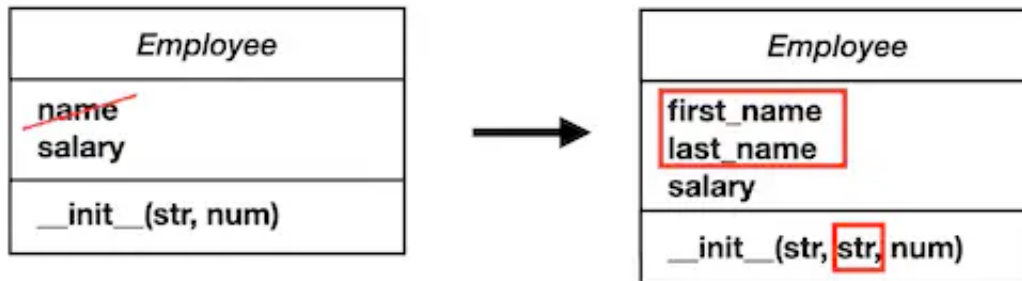
Sub requirement 2:

As a "staff manager,"

I want to "have a Staffing report including first name, last name, and job title,"
so that "I can track my staff."

Class refactoring

- The next step is to refactor our class design.



Code refactoring

- Finally, we refactor the code.
- We need to change the Employee class.

```
class Employee:
    def __init__(self, last_name, first_name, salary):
        self.first_name = first_name
        self.last_name = last_name
        self.salary = salary
```

Implementation

```
1 employees = [  
2     Manager("Schmidt", "Vera", 2000),  
3     Attendant("Norris", "Chuck", 1800),  
4     Attendant("Carrington", "Samantha", 1800),  
5     Cook("Jacketti", "Roberto", 2100),  
6     Mechanic("Dreisig", "Dave", 2200),  
7     Mechanic("River", "Tina", 2300),  
8     Mechanic("Rama", "Ringo", 1900),  
9     Attendant("Rainey", "Chuck", 1800),  
10 ]  
11  
12 def print_accounting_report():  
13     print("Accounting")  
14     print("=====  
15     for e in employees:  
16         print(f"{e.first_name} {e.last_name}, ${e.salary}")  
17  
18 def print_staffing_report():  
19     print("Staffing")  
20     print("=====  
21     for e in employees:  
22         print(f"{e.first_name} {e.last_name}, {e.job_title}")
```

```
Accounting  
=====  
Schmidt Schmidt, $2000  
Norris Norris, $1800  
Carrington Carrington, $1800  
Jacketti Jacketti, $2100  
Dreisig Dreisig, $2200  
River River, $2300  
Rama Rama, $1900  
Rainey Rainey, $1800  
  
Staffing  
=====  
Schmidt Schmidt, $2000  
Norris Norris, $1800  
Carrington Carrington, $1800  
Jacketti Jacketti, $2100  
Dreisig Dreisig, $2200  
River River, $2300  
Rama Rama, $1900  
Rainey Rainey, $1800
```

Checkpoint: Reporting

- Requirements evolve as business needs change
- Good design makes it easier to accommodate changes
- Separate reporting logic from data structure

IMPORTANT

- Always plan for future extensibility
- **Key Insight:** Design for change, not just current requirements.

We need to hide information: Encapsulation.

| What if we need a different format?

- What if someone needs to print out the names in a different format?

Vera Schmidt  **Schmidt, Vera**

- We can update the print code twice to meet this request.

```
def print_accounting_report():  
    print("Accounting")  
    print("=====")  
    for e in employees:  
        print(f'{e.first_name} {e.last_name}, ${e.salary}')  
  
def print_staffing_report():  
    print("Staffing")  
    print("=====")  
    for e in employees:  
        print(f'{e.first_name} {e.last_name}, ${e.job_title}')
```

Code Smell

- However, we need to change **two places**, not one, in this case.
- Also, the print methods should know the details (fields) of the Employee class.
 - It should be known that the Employee class has first_name and last_name.
 - It means it can change these fields.

- So, if the field name changes, the print code is impacted and needs updating.
- So we detect **two code smells**.

1: Duplicate code

2: Implementation details

The Root Problem

- The real problem is not that fields can be read from outside — that alone isn't always bad.
- The problem is that the **name-combining rule is duplicated** across reports.
- Each report depends directly on Employee's internal fields (`first_name`, `last_name`).

- So, the Employee class should own the responsibility of combining the name, not each report.
- It should expose that through a **method**, so reports don't need to know the internal storage.



Encapsulation

- We use Encapsulation (Information hiding) to address this issue.
- Key Principle: Hide implementation details, expose only what's necessary.

Tool box

✓ Objects & Classes

✓ Inheritance

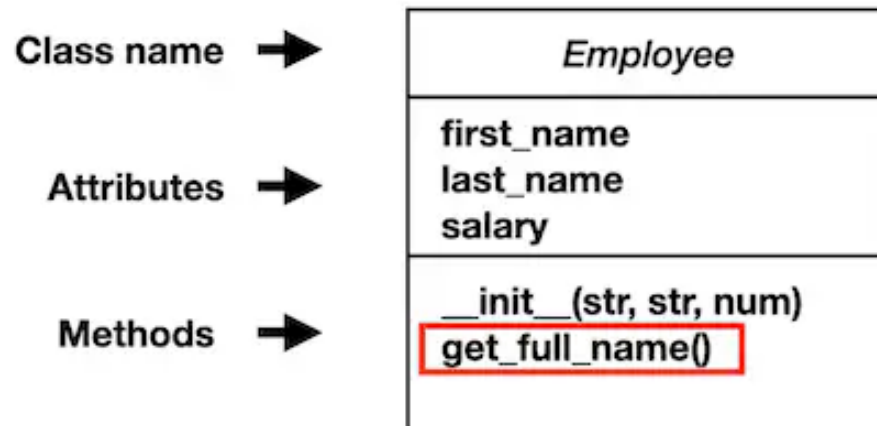
✓ Encapsulation

Polymorphism

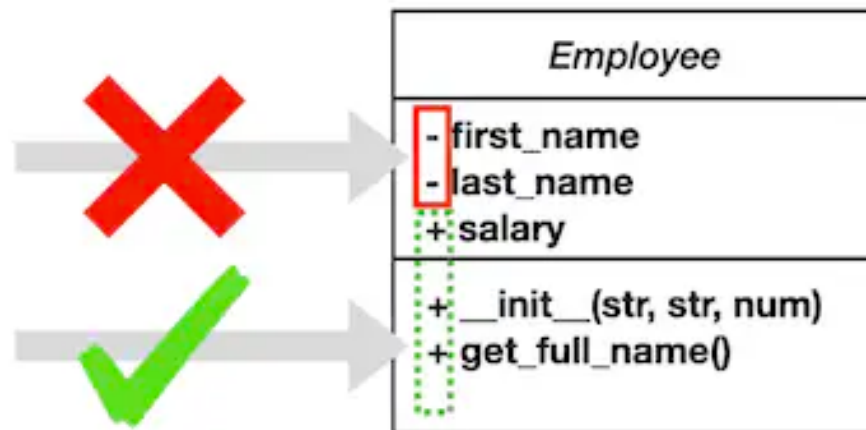
Composition

UML Design Changes

- We add a method `get_full_name()`.



- We hide the two fields by making them **private** (-).
- Instead, `get_full_name()` should be **public** (+) so any entity needing the name can use it.



Refactor code

- We can refactor the Employee class.

```
class Employee:
    def __init__(self, last_name, first_name, salary):
        self.first_name = first_name
        self.last_name = last_name
        self.salary = salary
    def get_full_name(self):
        return f"{self.last_name},{self.first_name}"
```

Python pitfall

- Python has no `public` / `private` keywords enforced by the language.
- Visibility is managed by **convention** instead.
- We prepend `_` to mark a field as **non-public by convention**.
- It's still technically accessible from outside, unlike a true UML `private` field.

Python's Non-Public Fields (by Convention)

- Because Python doesn't enforce access restrictions, it's easy to access internal state by accident, making bugs harder to track down.

```
class Employee:
    def __init__(self, last_name, first_name, salary):
        self._first_name = first_name # <-- mark as internal by convention
        self._last_name = last_name   # <-- mark as internal by convention
        self.salary = salary
    def get_full_name(self):
        return f"{self._last_name},{self._first_name}"
```

Implementation

- We can get the same results with a modification in the driver code.
- The two print methods can use `get_full_name()` to print the name **without knowing Employee's implementation details.**

- We get the same results as before.
- However, the Employee fields are isolated from external change.

```
1 def print_accounting_report():
2     print("Accounting")
3     print("=====")
4     for e in employees:
5         print(f"{e.get_full_name()}, ${e.salary}")
6
7 def print_staffing_report():
8     print("Staffing")
9     print("=====")
10    for e in employees:
11        print(f"{e.get_full_name()}, {e.job_title}")
```

```
Accounting
=====
Schmidt Schmidt, $2000
Norris Norris, $1800
Carrington Carrington, $1800
Jacketti Jacketti, $2100
Dreißig Dreißig, $2200
River River, $2300
Rama Rama, $1900
Rainey Rainey, $1800

Staffing
=====
Schmidt Schmidt, $2000
Norris Norris, $1800
Carrington Carrington, $1800
Jacketti Jacketti, $2100
Dreißig Dreißig, $2200
River River, $2300
Rama Rama, $1900
Rainey Rainey, $1800
```

Checkpoint: Encapsulation

- A class should hide its internal implementation details.
- It should expose a small set of **public methods** for other entities to use instead.
 - These public methods form the class's public API.

- We can manage complexity with encapsulation.
- **Benefits** of encapsulation include:
 - Single point of change for name formatting
 - Implementation details are hidden
 - The interface is stable and clean

Refactoring for Better Organization

- We find that the two reporting methods are beginning to get larger.

- So, we make two classes for reporting.



Creating Report Module

```
class AccountingReport:
    def print_accounting_report(self):
        print("Accounting")
        print("=====")
        # !!! Direct access to main.py's employees
        for e in employees:
            print(f"{e.get_full_name()}, ${e.salary}")
```

The AccountingReport uses the `employees` object to display information.

The same is true with the StaffingReport.

```
class StaffingReport:
    def print_staffing_report(self):
        print("Staffing")
        print("=====")
        for e in employees:
            print(f"{e.get_full_name()}, {e.job_title}")
```

Refactor main.py

- We need to refactor the main.py.
- We should create the `employees` for both of the report classes, but we forget to make the object.

```
from employee import Manager, Attendant, Cook, Mechanic
from reporting import AccountingReport, StaffingReport

# employees = ... # Oops ...
AccountingReport().print_accounting_report()
print() # empty line
StaffingReport().print_staffing_report()
```

Coupling

Dependency Issue

- This code will **crash** with `NameError: name 'employees' is not defined`.
- `reporting.py` never defines or imports `employees` — it just assumes the variable will exist at runtime.

- We call this a **hidden (global) dependency**:
`reporting.py` relies on an undeclared global.
- `main.py` can't provide it implicitly, since each Python module has its own namespace.



reporting.py

```
class AccountingReport:
    def print_accounting_report(self):
        print("Accounting")
        print("=====")
        for e in employees:
            print(f"{e.get_full_name()}, $
{e.salary}")
```

reporting.py

```
class StaffingReport:
    def print_staffing_report(self):
        print("Staffing")
        print("=====")
        for e in employees:
            print(f"{e.get_full_name()},
{e.job_title}")
```

The Cause of the Problem - Coupling

- `reporting.py` relies on an undeclared global variable, instead of receiving what it needs explicitly.
- Coupling itself is normal — objects must know about each other to collaborate.
- The real problem is that this dependency is **hidden**, not explicit.

Problem indicators

~~Duplicate code~~

~~Coupling~~

No Single Responsibility

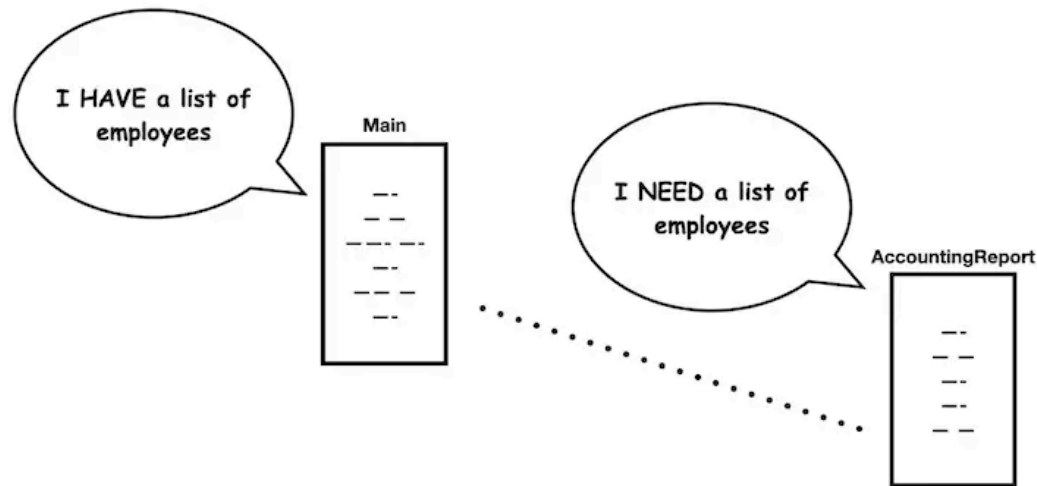
~~if/else~~

Warning: Hidden, tight coupling is a notorious code smell.

Solution - Dependency Injection

- This issue can be resolved by **dependency injection**.
- `main.py` has the `employees` object, but `reporting.py` tried to access it directly.

- We resolve this by **injecting the dependency** (the employees object) into the reporting.py classes.



Refactored Report Classes

- Now the dependency is injected.

```
1 class AccountingReport:
2     def __init__(self, emp_list):
3         # !!! Dependency is injected from outside
4         # !!! This class doesn't access the main.py any more
5         self._emp_list = emp_list
6
7     def print_accounting_report(self):
8         print("Accounting")
9         print("=====")
10        for e in self._emp_list:
11            print(f"{e.get_full_name()}, ${e.salary}")
12
13 class StaffingReport:
14     def __init__(self, emp_list):
15         self._emp_list = emp_list
16
17     def print_staffing_report(self):
18         print("Staffing")
19         print("=====")
20        for e in self._emp_list:
21            print(f"{e.get_full_name()}, {e.job_title}")
```

IMPORTANT

- The dependency is passed in through arguments.
- We no longer assume it exists elsewhere.
- This reduces coupling between classes.

Refactored main

- The main injects the employees into the report classes.

```
from reporting import AccountingReport, StaffingReport

employees = create_employees() # existing Employee objects

accounting_report = AccountingReport(employees) # injected dependency
accounting_report.print_accounting_report()
print()
staffing_report = StaffingReport(employees)
staffing_report.print_staffing_report()
```

UML Notation: Solid vs. Dashed Line

- **Dependency injection** is a design pattern; it is not the same thing as a UML relationship type.
- UML line styles describe how objects relate to each other.

Solid/Dashed Line in UML

- **Solid line** = an **association**; the dependency is kept as a field.
- **Dashed line with an arrow** = a **dependency**; the dependency is used briefly and not stored.

AccountingReport	————	Employee	:	association
AccountingReport	- - ->	Employee	:	dependency

UML Notation: Solid vs. Empty Diamond

- **Solid diamond** = stronger ownership; same lifecycle.
- **Empty diamond** = weaker ownership; independent lifecycle.

Car	◆	Engine	: composition
Company	◇	Employee	: aggregation

Final Takeaways

- **Hidden coupling** is a major code smell.
- Keep coupling low and make it explicit when needed.
- **Dependency injection** is a clear way to do that.

IMPORTANT

- Dependency injection does not remove the dependency; it makes it explicit.
- We turn hidden, hard-to-manage coupling into explicit, manageable coupling.
- `main.py` chooses what to pass in instead of hiding it inside the class.
- Key idea: pass dependencies in, instead of reaching into them directly.